

P4102R0

Container insertion and erasure should be allowed to relocate

Published Proposal, 2026-04-02

Author:

[Giuseppe D'Angelo](#)

Audience:

SG9, LEWG

Project:

ISO/IEC 14882 Programming Languages — C++, ISO/IEC JTC1/SC22/WG21

Abstract

We propose to change the type requirements of erasure and insertion operations in `vector`, `inplace_vector` and `deque`, enabling implementations to use relocation instead of assignment to shift elements, and extending these operations to types that are move-constructible but not move-assignable.

Table of Contents

- 1 Changelog**
- 2 Motivation and scope**
 - 2.1 How erasure and insertion currently work
 - 2.2 The relocation alternative
 - 2.3 Why the current specification blocks relocation
 - 2.4 When the two strategies diverge
 - 2.4.1 Types with write-through assignment
 - 2.4.2 PMR containers

2.5 Affected containers

3 Prior art

4 Design decisions

4.1 Requirements changes

4.1.1 `erase`

4.1.2 `insert` and `emplace`

4.1.3 Range insertion

4.2 Interaction with allocators

4.3 Exception safety

4.3.1 Current guarantees

4.3.2 `erase`: the basic guarantee

4.3.3 `insert` and `emplace`: the basic guarantee

4.3.4 Range insertion: the basic guarantee

4.3.5 The nothrow dispatch rule

4.4 The `is_replaceable` trait is not needed

4.5 Fallback: an opt-out mechanism

5 Impact on the Standard

5.1 Wording changes

5.2 ABI impact

5.3 Support for `const` element types

6 Future work

6.1 Relocation as a first-class concept

6.2 Algorithms

6.3 `vector<const T>`

6.4 Value semantics requirement for elements stored in containers

7 Proposed Wording

7.1 Sequence requirements

7.2 `vector` modifiers

7.3 `deque` modifiers

7.4 `inplace_vector` modifiers

8 Acknowledgements

References

Non-Normative References

§ 1. Changelog

- R0
 - First submission

§ 2. Motivation and scope

§ 2.1. How erasure and insertion currently work

The sequence containers `vector`, `inplace_vector` and `deque` store their elements in contiguous storage (or, in the case of `deque`, in blocks of contiguous storage). When an element is erased from the middle of such a container, the elements after the erased position must be shifted to fill the gap. Similarly, when an element is inserted in the middle, the existing elements must be shifted to make room.

The C++ Standard does not explicitly specify how these shifting operations are performed. For instance, the semantics of `vector::erase` are given by [\[sequence.reqmts\]](#) simply as "Erases the element pointed to by `q`", and [\[vector.modifiers\]](#) does not elaborate on the mechanism.

However, the *Complexity* clause in [\[vector.modifiers\]](#) states that:

Complexity: The destructor of `T` is called the number of times equal to the number of the elements erased, but the assignment operator of `T` is called the number of times equal to the number of elements in the vector after the erased elements.

This, combined with the requirement that `T` must be *Cpp17MoveAssignable* (but not necessarily *Cpp17MoveInsertable*), imposes a specific implementation strategy: the elements after the erased

range are moved to fill the gap by using move assignment, and then the trailing moved-from elements are destroyed. In pseudocode:

```
template <typename T>
constexpr auto vector<T>::erase(iterator first, iterator last)
{
    if (first == last)
        return last;

    // Step 1: move-assign leftward
    auto new_end = std::move(last, end(), first);

    // Step 2: destroy the tail
    std::destroy(new_end, end());

    // ... update bookkeeping ...

    return first;
}
```

A single-element `insert` operation, when no reallocation is needed and the insertion point is not at the end, is slightly more involved.

The last element must be move-constructed into the uninitialized storage past the end (since there is no existing object there to assign to). Then the remaining elements between the insertion point and the (former) last element are shifted rightward by move assignment. Finally, the new value is assigned over the moved-from element at the insertion point:

```
template <typename T>
constexpr auto vector<T>::insert(iterator pos, const T& value)
{
    if (size() == capacity()) { /* reallocate */ }
    else if (pos == end()) { /* insertion at the end */ }
    else {
        // Step 1: move-construct last element into uninitialized storage
        std::construct_at(std::to_address(end()), std::move(*(end() - 1)));

        // Step 2: move-assign the rest of the tail one position to the right
        std::move_backward(pos, end() - 1, end());

        // Step 3: assign the new value into the gap
    }
}
```

```

        *pos = value;

        // ... update bookkeeping ...
    }
}

```

Because these operations are specified in terms of assignment, the type requirements reflect this:

`erase` requires *Cpp17MoveAssignable* but does not require *Cpp17MoveInsertable* (that is, move constructibility via the allocator). `insert` requires both *Cpp17MoveAssignable* and *Cpp17MoveInsertable*.

The same pattern applies to `deque` ([[deque.modifiers](#)]) and `inplace_vector` ([[inplace.vector.modifiers](#)]), as well as to related operations like `emplace` (in the cases where existing elements need to be shifted).

§ 2.2. The relocation alternative

There is a different implementation strategy for shifting elements: instead of move-assigning each element to its new position, the implementation can *relocate* each element. A relocation consists of move-constructing the element at its new position and then destroying the original. In pseudocode:

```

template <typename T>
constexpr auto vector<T>::erase(iterator first, iterator last)
{
    if (first == last)
        return last;

    // Step 1: destroy erased elements
    std::destroy(first, last);

    // Step 2: relocate tail leftward
    std::uninitialized_relocate(last, end(), first);

    // Where uninitialized_relocate does, for each element (see P3516R2):
    //   std::construct_at(dst, std::move(*src));
    //   std::destroy_at(src);

    // ... update bookkeeping ...
}

```

```
    return first;
}
```

For the `insert` case:

```
template <typename T>
constexpr auto vector<T>::insert(iterator pos, const T& value)
{
    if (size() == capacity()) { /* reallocate */ }
    else if (pos == end()) { /* insertion at the end */ }
    else {
        // Step 1: relocate rightward
        std::uninitialized_relocate_backward(pos, end(), end() + 1);

        // Step 2: construct new element
        std::construct_at(std::to_address(pos), value);

        // ... update bookkeeping ...
    }
}
```

The relocation strategy is attractive for performance reasons. For *trivially relocatable* types (types for which relocation is equivalent to a bitwise copy) the entire shifting loop can be replaced by a single call to a trivial relocation primitive (such as the `std::trivially_relocate` proposed by [P2786R13]), which the implementation can carry out as an efficient low-level memory operation (such as `memmove`).

The overwhelming majority of types used in practice are trivially relocatable: scalar types, certain implementations of `std::string`, `std::vector<T>`, `std::unique_ptr<T>`, `std::shared_ptr<T>`, and most user-defined types composed of these. Trivial relocation instead of element-by-element move assignment and destruction provides order-of-magnitude performance improvements, especially for large containers.

The relocation-based algorithms (`std::uninitialized_relocate`, etc.) are the subject of [P3516R2]; they provide the building blocks that container implementations would use.

§ 2.3. Why the current specification blocks relocation

The current specification prevents implementations from using the relocation strategy for two reasons.

First, the type requirements differ. The relocation strategy requires move constructibility (that is, `Cpp17MoveInsertable` for allocator-aware containers), but `erase` currently only requires `Cpp17MoveAssignable`. A type that is move-assignable but not move-constructible could legally be stored in a vector and erased, but the relocation strategy would fail to compile for such a type.

In practice, this is not a real concern. A type with a working move assignment operator but a deleted move constructor is pathological. All common types satisfy both requirements, and it is hard to come up with a realistic use case for a *container* holding a type that is move-assignable but not move-constructible.

Second, and more importantly, **for certain types the two strategies produce *different observable results***. This is the core issue that this paper addresses.

§ 2.4. When the two strategies diverge

The two strategies (assignment-based and relocation-based) produce different results for types where move assignment has different semantics from destroy-then-move-construct. Following the terminology introduced by [P2786R13], we call such types *non-replaceable*: a type is replaceable if destroying an object and move-constructing a new one in its place is equivalent to move-assigning to it.

It is important to note that replaceability is a more specific property than reference semantics. A type can have reference semantics and still be replaceable. For example, `std::string_view` refers to external storage and does not own its data, but its move assignment simply copies the pointer and length; which is exactly what destroy-then-move-construct would do. `string_view` is replaceable, and the two shifting strategies produce identical results for it.

The types where the strategies diverge are the non-replaceable ones. There are two notable families.

§ 2.4.1. Types with write-through assignment

The canonical example is `std::tuple<int &>`, whose move assignment operator has write-through semantics: it assigns through the reference rather than rebinding it.

Consider the following example:

```
int a = 1, b = 2, c = 3;
std::vector<std::tuple<int &>> v = { {a}, {b}, {c} };

v.erase(v.begin()); // erase the first element
```

Under the **assignment strategy** (current specification), the following happens:

1. `v[1]` is move-assigned to `v[0]`. Because `tuple<int &>`'s move assignment has write-through semantics, this assigns the value of `b` (2) into the `int` referenced by `v[0]`, which is `a`. Now `a == 2`. `v[0]` still references `a`.
2. `v[2]` is move-assigned to `v[1]`. By the same logic, `b` gets the value of `c` (3). Now `b == 3`. `v[1]` still references `b`.
3. `v[2]` is destroyed (a no-op for references).

Result: `v` has two elements, `v[0]` references `a` and `v[1]` references `b`. The values of the external variables have been mutated: `a == 2`, `b == 3`, `c == 3`.

Under the **relocation strategy**, the following happens:

1. `v[0]` (referencing `a`) is destroyed (no-op).
2. `v[1]` is move-constructed into `v[0]`'s storage. The reference is *copied*, not written through: `v[0]` now references `b`. The original `v[1]` is then destroyed.
3. `v[2]` is move-constructed into `v[1]`'s storage. `v[1]` now references `c`. The original `v[2]` is then destroyed.

Result: `v` has two elements, `v[0]` references `b` and `v[1]` references `c`. The external variables are unchanged: `a == 1`, `b == 2`, `c == 3`.

The relocation strategy produces what most programmers would consider the "correct" result: erasing the first element should leave the remaining elements (references to `b` and `c`) intact, without mutating the referenced objects as a side effect.

The assignment strategy's behavior (silently overwriting `a`) is surprising, and arguably a design defect in how these types interact with container operations. Nevertheless, it is what the current Standard specifies.

Other types in this family include `std::pair` containing references, and user-defined types with analogous write-through assignment.

§ 2.4.2. PMR containers

A more practically significant family of non-replaceable types is PMR containers:

`std::pmr::vector<T>`, `std::pmr::string`, and other containers using `std::pmr::polymorphic_allocator`.

`pmr::polymorphic_allocator` does not provide a `propagate_on_container_move_assignment` member type, which causes `allocator_traits<pmr::polymorphic_allocator>` to report `propagate_on_container_move_assignment` as `false_type`. This means that when a PMR container is move-assigned, the target retains its own allocator. If the source and target use different memory resources, the move assignment cannot simply transfer ownership of the source's storage; instead, it must perform an element-wise move into the target's allocation. By contrast, move-constructing a PMR container from a source creates a fresh object that takes the source's memory resource and can take direct ownership of the source's storage.

This means that move assignment and destroy-then-move-construct are not equivalent for PMR containers: the allocator association of the resulting object may differ. PMR containers are therefore formally non-replaceable.

However, whether this non-replaceability actually matters in the context of container shifting depends on whether the elements within the vector share the same memory resource.

When all elements share the same memory resource (which is the common case, especially when using a `pmr::vector` that propagates its allocator to its elements via `uses_allocator` construction, for instance in a `pmr::vector<pmr::string>`), the allocators of all elements compare equal. In this scenario, move assignment *can* transfer ownership of the source's buffer (since the target's allocator can deallocate it), producing the same result as move construction. The two shifting strategies are effectively equivalent, even though the type is formally non-replaceable.

When elements use different memory resources (which requires deliberate setup, for instance by explicitly constructing elements with different memory resources) then the strategies diverge. Under the assignment strategy, each element retains the memory resource it was originally constructed with; the string *data* is moved (or copied) into the target's own allocation. Under the relocation strategy, the element is destroyed and a new one is move-constructed in its place; the new object takes the source's memory resource.

The key question is whether this difference in allocator association matters to the program. From a *value semantics* perspective, both strategies preserve the string's content: after erasing position 0, the string at position 0 holds the data that was previously at position 1, regardless of which memory resource backs it. The difference is only in *which allocator* the resulting object uses; this is arguably an implementation detail that programs should not depend on during element shifting.

§ 2.5. Affected containers

The containers affected by this proposal are:

- `std::vector`: the primary target, as it is the most commonly used contiguous container.
- `std::inplace_vector`: same storage model as `vector` (except that it cannot reallocate); same shifting operations.
- `std::deque`: deque uses blocks of contiguous storage; its `insert` and `erase` operations shift elements within and across blocks.

Node-based containers (`std::list`, `std::forward_list`) are not affected, as they do not shift elements. Associative and unordered containers are similarly unaffected (they are also node-based).

`std::hive` does not have the concept of "inserting at a position" (it inserts in a free position "somewhere"), and erasure is a $O(1)$ operation as it just adds the position to a free list; it is therefore also unaffected.

`std::basic_string` is not affected in practice, as its element type (`charT`) is always trivially relocatable, so the two strategies are indistinguishable.

§ 3. Prior art

[P2959R0] ("Container Element Replacement") explored the interaction between relocation and container operations, proposing a `container_replace_with_assignment` trait to allow type authors to control whether containers use assignment or construction-and-destruction for element replacement. This paper builds on that analysis but takes a different approach: rather than introducing a new trait, we change the type requirements so that the strategy follows from existing type properties.

[P3055R1] ("Relax wording to permit relocation optimizations in the STL") explored the same design space as this paper, proposing very similar relaxations to container operations to enable relocation-based shifting. P3055R1 was discussed in LEWG during the 2024-02-02 telecon, where there was consensus for more work in this direction (vote: 3/11/1/1/0). The author of P3055R1 has since stopped working on that paper; this paper continues in the same direction.

[P2786R13] ("Trivial Relocation for C++26") introduced the `is_replaceable` type property and proposed using it as a gate for container optimizations. As discussed in §4.4 [The `is\_replaceable` trait is not needed](#), we argue that this trait is unnecessary for the purpose of enabling relocation in container shifting operations. This paper is independent of, but complementary to, P2786: it addresses the prerequisite question of whether the type requirements and specification of insertion and erasure can be changed to allow relocation-based shifting. Without these changes, even if trivial relocation were standardized, implementations could not use it for these operations. If both this proposal and a trivial relocation proposal are adopted, implementations would be able to use trivial relocation for shifting elements when the element type is trivially relocatable.

[P1144R12] ("std::is_trivially_relocatable") is an alternative approach to trivial relocation. In that paper, the `relocatable` concept also includes a provision that "`T` models `relocatable` only if [...] If the expression `u2 = lv` is well-formed, then the expression has the same semantics as `u2.~T(); ::new ((void*)std::addressof(u2)) T(lv);`". This last property is indeed the `is_replaceable` type property of [P2786R13]. Once again, this provision exists in order to allow containers like `vector` to take advantage of trivial relocation; our proposal unlinks the relocation semantics from the behavior of containers (which we believe would still need the changes that we are proposing).

[P3516R2] ("Uninitialized relocate") proposes the `std::uninitialized_relocate` family of algorithms. These provide the building blocks that container implementations would use for the relocation strategy.

§ 4. Design decisions

§ 4.1. Requirements changes

This paper proposes to change the type requirements for `erase`, `insert`, `emplace`, and related operations, so that the shifting strategy available to the implementation follows from the properties of the element type.

The two shifting strategies (assignment-based and relocation-based) have *different* type requirements. The assignment strategy requires *Cpp17MoveAssignable*; the relocation strategy requires *Cpp17MoveInsertable* (move constructibility via the allocator). By specifying the operations in terms of *which requirements the type satisfies*, we give implementations freedom to use relocation where appropriate, while keeping the behavior constrained by the type's own properties.

The following table summarizes the proposed changes. Only the requirements related to shifting and permuting existing elements are shown; the requirements for the element being inserted (e.g. *Cpp17CopyInsertable*, *Cpp17EmplaceConstructible*) are unchanged.

Operation	Current requirement (shifting/permuting)	Proposed requirement (shifting/permuting)
<code>erase</code>	<i>Cpp17MoveAssignable</i>	<i>Cpp17MoveAssignable</i> or <i>Cpp17MoveInsertable</i>
<code>insert</code> (single)	<i>Cpp17MoveAssignable</i> and <i>Cpp17MoveInsertable</i>	<i>Cpp17MoveInsertable</i>

<code>emplace</code>	<i>Cpp17MoveAssignable</i> and <i>Cpp17MoveInsertable</i>	<i>Cpp17MoveInsertable</i>
<code>insert</code> (range) / <code>insert_range</code>	<i>Cpp17MoveAssignable</i> , <i>Cpp17MoveInsertable</i> , <i>Cpp17MoveConstructible</i> , <i>Cpp17Swappable</i>	<i>Cpp17MoveInsertable</i> , <i>Cpp17MoveConstructible</i>

Several observations on the proposed changes:

- They are **pure relaxations**: all proposed requirements are weaker than the current ones, so strictly more types satisfy them. Types that currently compile for these operations will continue to compile.
- When only one strategy is available (the type satisfies only one of the applicable requirements), the **implementation has no choice**: it must use that strategy.
- When multiple strategies are available, the implementation gains freedom to choose. This freedom is bounded by the exception safety constraints in [§ 4.3 Exception safety](#): when one strategy is nothrow and the other is potentially throwing, the nothrow strategy must be used. This may *require* implementations to switch away from their current strategy for certain types.
- Types that satisfy both requirements may see a change in *observable shifting behavior* (e.g. assignment side effects vs. construction side effects). We argue that this is acceptable, as discussed in [§ 2.4 When the two strategies diverge](#).

§ 4.1.1. `erase`

The requirement for `erase` is relaxed from *Cpp17MoveAssignable* to *Cpp17MoveAssignable* **or** *Cpp17MoveInsertable* (or both).

If the type satisfies only *Cpp17MoveAssignable*, the implementation must use the assignment strategy. This is the current behavior.

If the type satisfies only *Cpp17MoveInsertable*, the implementation must use the relocation strategy. This is a **new capability**: types that are move-constructible but not move-assignable (e.g. types with `const` or reference members) can now be erased from the middle of a `vector`. Currently, such an erase is ill-formed.

If the type satisfies both, the implementation may use either strategy, subject to the exception safety constraints in [§ 4.3 Exception safety](#).

§ 4.1.2. `insert` and `emplace`

The requirement for `insert` and `emplace` is relaxed from *Cpp17MoveAssignable* and *Cpp17MoveInsertable* to just *Cpp17MoveInsertable*.

Under the relocation strategy, the shifting of existing elements is performed entirely via move construction (into uninitialized storage) and destruction; therefore, no move assignment is needed. Similarly, the new element at the insertion point is *constructed* into the gap (via the allocator), rather than being *assigned* over a moved-from element. The *Cpp17MoveAssignable* requirement is therefore unnecessary.

If the type also satisfies *Cpp17MoveAssignable*, the implementation may use it for shifting as a quality-of-implementation matter. For instance, an implementation might prefer move assignment when it is `noexcept` and the move constructor is not (although note that even under the assignment strategy, the last element must be move-constructed past the end into uninitialized storage, so the nothrow status of the move constructor is relevant regardless of the shifting strategy chosen).

§ 4.1.3. Range insertion

Range insertion (`insert(pos, first, last)` and `insert_range`) currently requires *Cpp17MoveAssignable*, *Cpp17MoveInsertable*, *Cpp17MoveConstructible*, and *Cpp17Swappable*. We propose to require only *Cpp17MoveInsertable* and *Cpp17MoveConstructible*.

The *Cpp17MoveAssignable* requirement is dropped for the same reason as single-element insertion: the relocation strategy does not need it.

The *Cpp17Swappable* requirement is dropped because it is an implementation detail that has leaked into the interface. The requirement exists because some implementations handle range insertion by appending elements past the end and then calling `std::rotate` to move them into position (or, in the case of `deque`, by prepending elements and then reversing; see [LWG3742]). Both `rotate` and `reverse` are specified in terms of swaps, and therefore require *Cpp17Swappable*.

However, a rotation (or reversal) can equally be implemented as a sequence of relocations: move-construct one element into a temporary, relocate successive elements along the cycle, and move the temporary into the last position. This is exactly the same primitive (move construction followed by destruction) that this paper proposes for shifting.

The cycle leader is usually an object allocated on the stack, not into storage obtained from the allocator. Using `allocator_traits::construct` on non-allocator-owned storage is semantically inappropriate (and potentially wrong for allocators that customize `construct`, such as PMR allocators).

The temporary is therefore constructed directly via `T(rv)`, which requires *Cpp17MoveConstructible*. This requirement is already implied by *Cpp17MoveInsertable* for standard allocators, but is spelled out separately because it is needed even for allocators that customize `construct`. We therefore retain *Cpp17MoveConstructible* in the precondition.

Alternative implementation strategies may also avoid rotation entirely (e.g. libstdc++ accumulates the input range into a temporary container and then inserts from random-access iterators, cf. [here](#)).

If the type also satisfies *Cpp17MoveAssignable* and/or *Cpp17Swappable*, the implementation may use assignment-based shifting or swap-based rotation as a quality-of-implementation matter, subject to the nothrow dispatch rule (see [§4.3 Exception safety](#)).

§ 4.2. Interaction with allocators

For allocator-aware containers (`vector` and `deque`), the relocation strategy must use `allocator_traits::construct` and `allocator_traits::destroy` rather than placement new and direct destructor calls.

This is not a new pattern for these containers. Vector already uses `allocator_traits::construct` and `allocator_traits::destroy` when it *reallocates*: elements are move-constructed into the new buffer and destroyed in the old one, all through the allocator. The relocation strategy for insert/erase is the same operation, except that the source and destination are within the same buffer rather than across two different buffers.

In fact, the relocation strategy is *more* consistent with how allocator-aware containers work than the assignment strategy. Under the current specification, construction and destruction go through `allocator_traits`, but assignment (`operator=`) is called directly, bypassing the allocator entirely. The relocation strategy replaces that direct assignment call with construction and destruction, both of which go through the allocator.

Note that the relocation algorithms proposed in [\[P3516R2\]](#) (`std::uninitialized_relocate`, etc.) are not allocator-aware: they use `std::construct_at` and `std::destroy_at` directly. An allocator-aware container implementation using the relocation strategy would need to perform the construction and destruction steps through `allocator_traits::construct` and `allocator_traits::destroy` itself, rather than delegating to the P3516 algorithms. This is not novel; both libstdc++ and libc++ have internal versions of the `uninitialized_*` algorithms that are allocator-aware.

For `inplace_vector`, which does not use an allocator, the P3516 algorithms could be used directly.

§ 4.3. Exception safety

§ 4.3.1. Current guarantees

The three affected containers (`vector`, `inplace_vector`, and `deque`) currently provide nothrow guarantees for `erase`, `insert`, `emplace`, and related operations, conditional on the nothrow-ness of the element type's move operations. For `erase`, the nothrow guarantee depends solely on the move assignment operator. For `insert` and `emplace`, it depends on both the move constructor (used for the boundary element, which is always move-constructed into uninitialized storage regardless of the shifting strategy) and the move assignment operator (used for the remaining elements).

Under the relocation strategy, the move constructor takes over the shifting role that move assignment currently plays. The exception safety wording must therefore be generalized to cover both strategies. The overarching goal is to ensure that the proposed changes do not regress the exception safety guarantees currently provided; this is formalized by the nothrow dispatch rule (§ 4.3.5 [The nothrow dispatch rule](#)).

§ 4.3.2. `erase`: the basic guarantee

Both strategies provide the basic exception safety guarantee (the container is left in a valid state), but the postconditions differ when an exception is thrown mid-shift.

For `vector` and `inplace_vector`, the shifting always proceeds in one direction (leftward from `pos` to `end()`).

Under the **assignment strategy**, if an exception is thrown mid-shift, the container retains its original size. Some element somewhere in `[pos, end())` is in a moved-from (but valid) state; its value is indeterminate.

Under the **relocation strategy**, the erase operation first destroys the elements to be erased, leaving a window of uninitialized storage. Then the tail elements are relocated leftward to close the window. If an exception is thrown during relocation, the container cannot be left with a window of uninitialized storage in the middle. The elements that still need to be relocated must be destroyed (the already-relocated elements in their new positions could be kept, but the P3516 algorithms destroy everything because the caller cannot determine how many were transferred before the exception). Either way, the container's size has decreased; all remaining elements are fully determinate.

For `deque`, the analysis is similar but the affected range is wider. `deque::erase` may shift either the tail or the prefix (choosing whichever is shorter), so under the assignment strategy, the element left in a moved-from state may be anywhere in the container, not just in `[pos, end())`. Correspondingly,

under the relocation strategy, if the shift throws, more elements may be destroyed and the container may be left significantly shorter.

One can argue that the relocation-based postcondition is actually preferable: the container is shorter, but all of its remaining elements are fully determinate. The assignment-based postcondition preserves the container's size but leaves some element in an indeterminate state.

§ 4.3.3. `insert` and `emplace`: the basic guarantee

This analysis covers the case where no reallocation is needed and the insertion point is not at the end.

Insertion has a major asymmetry compared to `erase`: even under the assignment strategy, the boundary element must always be move-constructed into previously uninitialized storage. The nothrow guarantee for `insert` and `emplace` has therefore always depended on the nothrow-ness of the move constructor, not just the move assignment operator.

For `vector` and `inplace_vector`, shifting is rightward. Under the **assignment strategy**, each step move-assigns one element one position to the right, simultaneously "restoring" the destination and leaving the source moved-from. If the operation throws, the container's size has already increased by one (the boundary element was successfully move-constructed at the new last slot before shifting began), and some element somewhere in `[pos, end())` is in a moved-from state.

Under the **relocation strategy**, relocation maintains exactly N live elements throughout the shifting phase (each step constructs the new position then immediately destroys the source). If an exception is thrown mid-relocation, the already-relocated elements must be destroyed to restore invariants; the container ends up shorter than its pre-operation size, with all surviving elements determinate.

In both cases, placement of the new element happens after all shifting completes, but the failure modes differ. Under the assignment strategy, `pos` holds a moved-from object; placing the new element means copy-assigning, move-assigning, or (for `emplace`) constructing a temporary and move-assigning it over that slot. A throw leaves `*pos` in a moved-from state; that is, the same failure mode as a throw during shifting. Under the relocation strategy, `pos` holds no live object after shifting; the new element is constructed directly into that storage. A throw now leaves a hole at `pos`, and the implementation must destroy the tail `[pos+1, end())` to restore a valid (shorter) container.

For `deque`, `insert` may shift from either end; under the assignment strategy, the element left in a moved-from state may be anywhere in the container. Under the relocation strategy, if the shift throws, the container may be left significantly shorter.

§ 4.3.4. Range insertion: the basic guarantee

Implementations have wide latitude in how they implement range insertion. They may shift existing elements to create a window and then construct new elements into it; or they may append at the end and then permute the sequence into place; or use other strategies entirely. Each approach has its own failure modes.

If an exception is thrown by an operation that is unrelated to element move, copy, or assignment (such as an iterator operation or a buffer allocation), the insertion can typically be abandoned before any elements are disturbed, providing a strong guarantee. This case is unaffected by this proposal.

If an exception is thrown during the shifting phase under the **assignment strategy**, the result is similar to single `insert`: the container's size may already have increased, and some element somewhere in the container is in a moved-from state.

Under the **relocation strategy**, an exception during shifting or permutation causes the affected elements to be destroyed, leaving the container shorter than its pre-operation size, with all surviving elements fully determinate. Again, this is analogous to single `insert` and `erase`.

The key goal of this paper is that these changes do **not regress** exception safety relative to the current specification. The nothrow dispatch rule (§4.3.5 [The nothrow dispatch rule](#)) ensures that when a nothrow strategy is available, it is used, preventing any regression for types whose operations are currently nothrow. Formalizing the exact postconditions in the wording is left to future work.

§ 4.3.5. The nothrow dispatch rule

When both *Cpp17MoveAssignable* and *Cpp17MoveInsertable* are satisfied, the implementation has a choice of strategy. This choice must not cause a regression in exception safety.

Consider a type `T` that is nothrow move-assignable but has a potentially throwing move constructor. A concrete example is `std::list` on some implementations (e.g. MSVC), where the move constructor may allocate a sentinel node and can therefore throw. Under the current specification, `vector<std::list<int>>::erase` is guaranteed not to throw during the shifting phase, because the shifting is done by noexcept move assignment. If an implementation switched to the relocation strategy, the shifting would use the potentially-throwing move constructor, and the erase could throw mid-shift. This would be a regression in exception safety.

We propose the following rule: **when both strategies are available and one is nothrow while the other is potentially throwing, the implementation must use the nothrow strategy.**

In concrete terms for `erase`:

- If the assignment strategy is nothrow but the relocation strategy may throw: the implementation must use assignment.
- If the relocation strategy is nothrow but assignment may throw: the implementation must use relocation.
- If both strategies are nothrow: the implementation may use either (this is the common case).
- If both strategies may throw: the implementation may use either; neither offers a stronger guarantee.

Note that the second case (nothrow relocation but throwing assignment) *requires* implementations to switch to relocation for such types. This is not merely a permission; it is a mandate driven by exception safety.

The motivation for this rule differs between `erase` and insertion. For `erase`, the rule is required in order not to regress: the existing *Throws* clauses in `[vector.modifiers]`, `[deque.modifiers]`, and `[in-place.vector.modifiers]` name the assignment operator as the sole exception source during shifting. Without this rule, a "naïve" switch to relocation on a type such as `std::list` would violate that clause. The rule therefore *preserves* an existing codified guarantee.

For `insert`, `emplace`, and range insertion, the existing standard does not codify a nothrow-on-move guarantee for middle insertion. Nothing compels us to codify a dispatch rule at all. We codify it on the shifting phase anyway, to prevent implementations from silently regressing observable nothrow behavior of existing code. Here the rule introduces new normative text rather than preserving an existing guarantee.

Note that for single `insert` and `emplace`, the move constructor is load-bearing regardless of which shifting strategy is used: the boundary element must always be move-constructed past the end into uninitialized storage. A full nothrow guarantee therefore additionally requires the move constructor and the new-element constructor to be nothrow, not just the shifting operation.

For range insertion, the rule does not extend beyond shifting. Other sub-operations (permutation, if used; iterator access; buffer allocation; and construction of new elements into the gap) are not subject to the nothrow dispatch rule. Range insertion admits many implementation strategies and provides only the basic guarantee today; this paper does not attempt to codify a broader normative rule for it.

§ 4.4. The `is_replaceable` trait is not needed

The "Trivial Relocation for C++26" proposal ([P2786R13]) introduced the "replaceable" type property to capture exactly the property relevant here: whether destroying an object and constructing a new one in its place is equivalent to assigning to it. The trait was introduced out of concern that the library

needed an explicit opt-in (a language-level property) before implementations could be allowed to use relocation (and, transitively, trivial relocation) for container operations such as insertion and erasure.

This paper takes a different stance. The reason implementations could not use relocation for these operations is not a missing trait, but an over-specification of their behavior. The current wording constrains implementations to use assignment by specifying complexity in terms of calls to the assignment operator. By changing the type requirements and relaxing this over-specification, we give implementations the freedom to use relocation (and, in the future, trivial relocation) without requiring any new language-level machinery. The choice of strategy is driven mechanically by existing type properties (*Cpp17MoveAssignable*, *Cpp17MoveInsertable*, and their **noexcept** status), not by a new trait.

We therefore argue that the **is_replaceable** trait is unnecessary for this purpose. This is because:

- **The set of types where this matters in practice is small.** The types for which the two strategies produce observably different results are non-replaceable types: those where move assignment is not equivalent to destroy-then-move-construct. This includes types with write-through assignment semantics (like `tuple<int &>`) and, formally, PMR containers. However, as discussed in the Motivation section, the `tuple<int &>` case is exotic and already problematic in containers, while the PMR case only diverges when elements use different memory resources, which is an uncommon configuration. In the typical PMR setup (all elements sharing one resource), the strategies are equivalent.
- **Containers already assume value semantics.** The Standard's container requirements have always been designed around types with value semantics. For instance, the *Cpp17MoveConstructible* requirement states that after move construction, "the **value** of `u` is equivalent to the value of `rv` before the construction" (emphasis ours). The concept of equivalence in terms of *values* (rather than identity or resource association) pervades the container requirements. Allowing relocation is consistent with this assumption.
- **No new machinery is required.** If we do not gate the optimization on a trait, no new type traits, no new concepts, and no new language features are needed. The strategy dispatch is based entirely on existing named requirements and **noexcept** properties, which implementations already query. This makes the proposal simpler to specify, simpler to implement, and simpler to teach.
- **Laying the groundwork for trivial relocation in the language.** P2786 was voted into C++26 but was subsequently removed and deferred. One lesson from that experience is that attempting to solve the entire trivial relocation problem in a single proposal (the language aspects, the library algorithms, the container optimizations) makes the design space too large and the consensus too difficult to achieve.

This paper takes a more incremental approach, similar to [\[P3516R2\]](#) ("Uninitialized algorithms for relocation"). P3516 proposes relocation algorithms that are useful today on their own merits,

but that will be further optimized once trivial relocation is standardized. Similarly, this paper proposes changing the type requirements of container operations so that they can use relocation today, and will benefit from trivial relocation in the future.

By adopting these proposals independently, we lay the groundwork so that when trivial relocation is discussed again, there will be no need to re-litigate questions like "should containers be allowed to use relocation?" or "do we need relocation algorithms?". Those pieces will already be in place, and the discussion can focus on the core language feature.

§ 4.5. Fallback: an opt-out mechanism

If SG9/LEWG finds the requirement changes proposed above to be too aggressive, a fallback position is available: an opt-out mechanism similar to what [\[P2959R0\]](#) proposed with its `container_replace_with_assignment` trait.

Under this approach, the library is allowed to use the relocation strategy by default for all types, but a type author can explicitly opt out by specializing a trait:

```
// Hypothetical opt-out mechanism:  
template<>  
struct std::use_assignment_for_container_shift<MyNonReplaceableType>  
    : std::true_type {};
```

Types that specialize this trait to `true_type` would be guaranteed to always use the assignment strategy.

An opt-out (rather than opt-in) is the correct default here: the vast majority of types benefit from the optimization and do not care which strategy is used. Only the rare non-replaceable type would need to opt out.

This fallback adds complexity: a new trait, a new concept for users to learn, a new specialization point. We believe it is unnecessary, but it is available as a compromise if the blanket permission does not achieve consensus.

§ 5. Impact on the Standard

§ 5.1. Wording changes

Summarizing, the proposed changes affect the following areas:

- [\[container.reqmts.general\]](#) / [\[sequence.reqmts\]](#): the container requirements tables, which specify the type requirements for operations like `erase` and `insert`. The `erase` row would change from requiring `Cpp17MoveAssignable` to requiring `Cpp17MoveAssignable` **or** `Cpp17MoveInsertable`. The `insert` and `emplace` rows would drop the `Cpp17MoveAssignable` requirement, keeping only `Cpp17MoveInsertable` (for shifting). The range insertion rows would additionally drop `Cpp17MoveAssignable` and `Cpp17Swappable`, retaining `Cpp17MoveConstructible` (needed for the cycle-leader temporary in relocation-based rotation).
- [\[vector.modifiers\]](#), [\[deque.modifiers\]](#), [\[inplace.vector.modifiers\]](#): the *Effects*, *Complexity*, and *Throws* clauses for `erase`, `insert`, and `emplace`. The *Effects* would state that elements may be shifted either by move assignment or by relocation (move construction followed by destruction), depending on which requirements the type satisfies (see [§ 4.1 Requirements changes](#)). For range insertion, the *Effects* would additionally state that elements may be permuted using relocation instead of swaps. The *Complexity* clauses currently count operations in terms of assignments and destructor calls; these would need to be generalized to count either assignments or move constructions plus destructions. The *Throws* clauses would be generalized to cover exceptions from either strategy, with the nothrow dispatch rule from [§ 4.3.5 The nothrow dispatch rule](#).

§ 5.2. ABI impact

This proposal has no ABI impact. The choice of shifting strategy is an implementation detail that does not affect the binary interface of any container.

§ 5.3. Support for `const` element types

By enabling the relocation strategy, this proposal makes it possible, in principle, to store `const T` elements in sequence containers. The relocation strategy shifts elements by construction and destruction rather than assignment; `const` objects support construction and destruction but not assignment, so relocation is the only viable shifting strategy for `const` element types.

For `inplace_vector`, which is not allocator-aware and constructs and destroys elements directly (without going through `allocator_traits`), this paper enables `inplace_vector<const T>` for any CopyConstructible `T`. Operations such as `erase` and middle `insert` become well-formed.

For `vector` and `deque`, the type requirements proposed here are also satisfied by `const T` (copy construction serves in place of move construction). However, `vector<const T>` and `deque<const T>` remain ill-formed for a different reason: the `Cpp17Allocator` requirements do not permit cv-qualified element types. Resolving this requires changes to the allocator requirements, which is outside the scope of this paper (see §6.3 `vector<const T>`).

§ 6. Future work

§ 6.1. Relocation as a first-class concept

This paper uses *Cpp17MoveInsertable* as the requirement for the shifting phase of `erase`, `insert`, `emplace`, and related operations, because no formal named requirement for relocation exists in the standard today. This is a pragmatic stand-in: relocation ("construct-and-destroy" as a paired operation) and move construction (construct, leaving the source alive) express different operations and have different use sites within these container operations.

[P3516R2] is already moving in this direction at the algorithm level, introducing `relocate_at` as a new primitive and high-level algorithms which are built on top of that. A natural follow-up (to both this paper and that paper) would be to introduce a *Cpp17Relocatable* named requirement, currently equivalent to *Cpp17MoveInsertable*, but better expressing the intent of a "pairwise" operation; as well as to reword the container requirements to use it where shifting is the intended operation. Under this rewording:

- `erase` would require *Cpp17Relocatable* (shifting only);
- `insert` and `emplace` would require *Cpp17Relocatable* for the shifting phase and/or the reallocation in new memory, plus *Cpp17CopyInsertable*, *Cpp17MoveInsertable*, or *Cpp17EmplaceConstructible* for constructing the new element (depending on the overload);
- range insertion would require *Cpp17Relocatable* for both the shifting phase and any permutation of existing elements, plus *Cpp17EmplaceConstructible* (from the range element type) for constructing the new elements.

This split is already latent in the current design: the shifting role and the new-element-construction role are served by different operations on different elements, and naming them separately makes the intent precise.

The reword would have no behavioral impact on existing types, since every type that is *Cpp17MoveInsertable* is also *Cpp17Relocatable* today. If and when relocate-only types become possible through trivial relocation or other mechanisms, the container requirements would already be expressed in the right terms to directly support them.

The most significant benefit, however, would concern exception safety. The exception specifications in [\[vector.modifiers\]](#) and elsewhere are currently phrased conditionally on the nothrow-ness of the move constructor. When the shifting operation is logically a relocation, those conditions should instead refer to the nothrow-ness of the relocation operation. This distinction matters because a type may be nothrow (trivially) relocatable while having a throwing move constructor. Once *Cpp17Relocatable* is available, the exception specifications for `erase`, `insert`, and related operations can be amended accordingly, making the nothrow guarantees available to a broader class of types without any change to the behavioral rules.

§ 6.2. Algorithms

The same argument applies to algorithms that shift elements within a range, such as `std::remove` and `std::unique`. These algorithms are currently specified in terms of move assignment (and require *MoveAssignable*). Allowing them to use relocation instead would enable the same class of optimizations.

Similarly, `std::rotate` and `std::reverse` are specified in terms of swaps (and require *Swappable*). This paper already argues that *containers* can use relocation-based rotation internally for range insertion (see [§4.1.3 Range insertion](#)), but the `std::rotate` and `std::reverse` algorithms themselves could also benefit from being allowed to use relocation instead of swaps. Additionally, when operating on iterators over contiguous storage of a trivially relocatable type, `iter_swap` could be specialized to perform a bitwise swap of the underlying storage. This is left to a future paper.

§ 6.3. `vector<const T>`

As noted in [§5.3 Support for const element types](#), the sole remaining blocker for `vector<const T>` and `deque<const T>` is the *Cpp17Allocator* requirement that element types be cv-unqualified. With this paper in place, relaxing that constraint would be sufficient to make `vector<const T>` fully usable. That relaxation is outside the scope of this paper and is left to future work.

§ 6.4. Value semantics requirement for elements stored in containers

This paper touches on a broader question: should the Standard formally require that container element types have value semantics? Such a requirement would resolve not only the issue addressed here, but also related issues with `swap`, `sort`, and other operations that behave surprisingly for types with reference semantics. This is a larger discussion that deserves its own paper.

§ 7. Proposed Wording

Changes are relative to [N5032].

§ 7.1. Sequence requirements

Modify the *Preconditions* of each of the following operations in [sequence.reqmts]:

- `a.erase(q):`

For `vector`, `inplace_vector`, and `deque`, `T` is *Cpp17MoveAssignable* or *Cpp17MoveInsertable* into `X`.

- `a.erase(q1, q2):`

For `vector`, `inplace_vector`, and `deque`, `T` is *Cpp17MoveAssignable* or *Cpp17MoveInsertable* into `X`.

- `a.emplace(p, args):`

`T` is *Cpp17EmplaceConstructible* into `X` from `args`. For `vector`, `inplace_vector`, and `deque`, `T` is also *Cpp17MoveInsertable* into `X` ~~and *Cpp17MoveAssignable*~~.

- `a.insert(p, t):`

`T` is *Cpp17CopyInsertable* into `X`. For `vector`, `inplace_vector`, and `deque`, `T` is also *Cpp17CopyAssignable* or *Cpp17MoveInsertable* into `X`.

- `a.insert(p, rv):`

`T` is *Cpp17MoveInsertable* into `X`. ~~For `vector`, `inplace_vector`, and `deque`, `T` is also *Cpp17MoveAssignable*.~~

- `a.insert(p, n, t):`

~~`T` is *Cpp17CopyInsertable* into `X` and *Cpp17CopyAssignable*.~~ `T` is *Cpp17CopyInsertable* into `X`. For `vector`, `inplace_vector`, and `deque`, `T` is also *Cpp17CopyAssignable* or *Cpp17MoveInsertable* into `X`. For other sequence containers, `T` is also *Cpp17CopyAssignable*.

- `a.insert(p, i, j):`

T is *Cpp17EmplaceConstructible* into X from *i. For vector, inplace_vector, and deque, T is also *Cpp17MoveInsertable* into X, and T meets the *Cpp17MoveConstructible* ; ~~*Cpp17MoveAssignable*, and *Cpp17Swappable* ([swappable.requirements])~~ requirements. Neither i nor j are iterators into a.

- `a.insert_range(p, rg):`

T is *Cpp17EmplaceConstructible* into X from *ranges::begin(rg). For vector, inplace_vector, and deque, T is also *Cpp17MoveInsertable* into X, and T meets the *Cpp17MoveConstructible* ; ~~*Cpp17MoveAssignable*, and *Cpp17Swappable* ([swappable.requirements])~~ requirements. rg and a do not overlap.

- `a.prepend_range(rg):`

T is *Cpp17EmplaceConstructible* into X from *ranges::begin(rg). For deque, T is also *Cpp17MoveInsertable* into X, and T meets the *Cpp17MoveConstructible* ; ~~*Cpp17MoveAssignable*, and *Cpp17Swappable* ([swappable.requirements])~~ requirements.

§ 7.2. vector modifiers

Modify [\[vector.modifiers\]](#) as follows.

For `insert` and `emplace`, add to *Remarks*:

When no reallocation occurs and the insertion point is not at the end, elements are shifted to make room. The implementation may shift elements either by move assignment or by relocation (move construction into the new position followed by destruction of the source). If T is both *Cpp17MoveAssignable* and *Cpp17MoveInsertable*, the implementation may use either strategy. If exactly one of `is_nothrow_move_assignable_v<T>` and `is_nothrow_move_constructible_v<T>` is true, the nothrow strategy shall be used for shifting. For range insertion, the implementation may also permute existing elements using relocation instead of swaps.

For `erase`:

- Modify *Throws*:

Nothing unless an exception is thrown by the assignment operator or move assignment operator of T, or by the copy constructor or move constructor of T.

- Modify *Complexity*:

The destructor of T is called the number of times equal to the number of the elements erased, but the ~~assignment operator of T is called the number of times equal to the number of elements in the vector after the erased elements~~ number of shift operations on T equals the number of elements after the erased elements. Each shift operation consists of either a move assignment or a relocation (a move construction into the new position followed by destruction of the source).

- Add new *Remarks*:

If T is both *Cpp17MoveAssignable* and *Cpp17MoveInsertable*, the implementation may use either move assignment or relocation for shifting. If exactly one of *is_nothrow_move_assignable_v<T>* and *is_nothrow_move_constructible_v<T>* is true, the nothrow strategy shall be used.

§ 7.3. deque modifiers

Modify [\[deque.modifiers\]](#) as follows.

For `insert` and `emplace`, add to *Remarks*:

The implementation may shift elements either by move assignment or by relocation (move construction into the new position followed by destruction of the source). If T is both *Cpp17MoveAssignable* and *Cpp17MoveInsertable*, the implementation may use either strategy. If exactly one of *is_nothrow_move_assignable_v<T>* and *is_nothrow_move_constructible_v<T>* is true, the nothrow strategy shall be used for shifting. For range insertion, the implementation may also permute existing elements using relocation instead of swaps.

For `erase`:

- Modify *Throws*:

Nothing unless an exception is thrown by the ~~assignment operator~~ assignment operator or move assignment operator of T, or by the copy constructor or move constructor of T.

- Modify *Complexity*:

The number of calls to the destructor of T is the same as the number of elements erased, but the number of ~~calls to the assignment operator of T is~~ shift operations on T (each being a move assignment or a relocation) is no more than the lesser of the number of elements before the erased elements and the number of elements after the erased elements.

- Add new *Remarks*:

If T is both *Cpp17MoveAssignable* and *Cpp17MoveInsertable*, the implementation may use either move assignment or relocation for shifting. If exactly one of *is_nothrow_move_assignable* v<T> and *is_nothrow_move_constructible* v<T> is true, the nothrow strategy shall be used.

§ 7.4. `inplace_vector` modifiers

Modify [\[inplace_vector.modifiers\]](#) as follows.

For `insert` and `emplace`, add to *Remarks*:

When the insertion point is not at the end, elements are shifted to make room. The implementation may shift elements either by move assignment or by relocation (move construction into the new position followed by destruction of the source). If T is both *Cpp17MoveAssignable* and *Cpp17MoveInsertable*, the implementation may use either strategy. If exactly one of *is_nothrow_move_assignable* v<T> and *is_nothrow_move_constructible* v<T> is true, the nothrow strategy shall be used for shifting. For range insertion, the implementation may also permute existing elements using relocation instead of swaps.

For `erase`:

- Modify *Throws*:

Nothing unless an exception is thrown by the assignment operator or move assignment operator of T , or by the copy constructor or move constructor of T.

- Modify *Complexity*:

The destructor of T is called the number of times equal to the number of the elements erased, but the assignment operator of T is called the number of times equal to the number of elements after the erased elements. number of shift operations on T equals the number of elements after the erased elements. Each shift operation consists of either a move assignment or a relocation (a move construction into the new position followed by destruction of the source).

- Add new *Remarks*:

If T is both *Cpp17MoveAssignable* and *Cpp17MoveInsertable*, the implementation may use either move assignment or relocation for shifting. If exactly one of *is_nothrow_move_assignable_v<T>* and *is_nothrow_move_constructible_v<T>* is true, the nothrow strategy shall be used.

§ 8. Acknowledgements

Thanks to KDAB for supporting this work.

Thanks to Alisdair Meredith for [P2959R0], which explored the interaction between relocation and container operations. Thanks to Arthur O'Dwyer for the foundational work in [P3055R1], upon which this proposal is based. Thanks to Louis Dionne for the collaboration on [P3516R2].

All remaining errors are ours and ours only.

§ References

§ Non-Normative References

[LWG3742]

Casey Carter. *deque::prepend_range needs to permute*. C++23. URL: <https://wg21.link/lwg3742>

[N5032]

Thomas Köppe. *Working Draft, Standard for Programming Language C++*. 15 December 2025. URL: <https://wg21.link/n5032>

[P1144R12]

Arthur O'Dwyer. *std::is_trivially_relocatable*. 15 October 2024. URL: <https://wg21.link/p1144r12>

[P2786R13]

Pablo Halpern, Joshua Berne, Corentin Jabot, Pablo Halpern, Lori Hughes. *Trivial Relocatability For C++26*. 14 February 2025. URL: <https://wg21.link/p2786r13>

[P2959R0]

Alisdair Meredith. *Container Relocation*. 15 October 2023. URL: <https://wg21.link/p2959r0>

[P3055R1]

Arthur O'Dwyer. *Relax wording to permit relocation optimizations in the STL*. 12 February 2024. URL: <https://wg21.link/p3055r1>

[P3516R2]

Louis Dionne, Giuseppe D'Angelo. *Uninitialized algorithms for relocation*. 16 May 2025. URL: <https://wg21.link/p3516r2>